

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Final
Semester: Spring 2025

Duration: 1 Hour 40 Min
Full Marks: 40

CSE 470: Software Engineering

SET B

Name: Sabbir Ahmad

ID: 24141266

Section: 07

NB: For Questions 2 and 3, you have been provided with both JAVA & equivalent Python Code for better understanding. Use any one of the code versions to solve these questions.

Q1. [CO3] Mayor Wilson Fisk decides to upgrade the public safety infrastructure of NYC by deploying a smart surveillance platform named SNY. Wilson hires Harold Finch, a well-known software architect, to design and implement this surveillance platform along with a centralized monitoring system known as the Security Control Center (SCC). People of NYC need to register for this platform to get important security updates from SCC in real-time. To ensure operational integrity and consistency, Harold suggests that only one global access point of the control center should exist during the application's lifetime to prevent resource overload.

- a. Mention the design pattern/s needed to implement this system. [2]
- ☒ b. Implement the required design pattern/s in your preferred coding language. [8]
- ☒ c. Could we implement the Singleton pattern in the PeopleNYC class? Why or why not? [2]

Q2. [CO4]

- ☒ a. Calculate the **Specialization Index (SIX)** value of the class **Intern** from the code snippet below. Write the **formula** and its values for the calculation. [3]
- ☒ b. Draw the Control Flow Graph (CFG) for the **calculatePerformance** method in the **StaffMember** class from the code snippet below. [6]
- ☒ c. Find the Cyclomatic Complexity Number for the CFG that you have drawn for the **calculatePerformance** method in the **StaffMember** class from the code snippet below, using all three formulae. [2]
- ☒ d. How many independent paths are possible for a CFG? Find **three** independent paths for the CFG that you have drawn for the **calculatePerformance** method in the **StaffMember** class from the code snippet below. [3]
- ☒ e. Write two differences between **White Box Testing** and **Grey Box Testing**. [2]

JAVA	PYTHON
<pre>abstract class Employee { protected String name; protected int employeeId; public Employee(String name, int employeeId) { this.name = name; this.employeeId = employeeId; } public abstract double calculateSalary(); public String generateReport() { return "Name: " + name; } }</pre>	<pre>class Employee: def __init__(self, name, employee_id): self.name = name self.employee_id = employee_id def calculateSalary(self): pass def generateReport(self): return f"Name: {self.name}"</pre>

```

    }
}

class StaffMember extends Employee {
    protected int baseSalary;

    public StaffMember(String name, int employeeId, double baseSalary) {
        super(name, employeeId);
        this.baseSalary = baseSalary;
    }

    public double calculatePayment(int Experience){
        return baseSalary+Experience * 100.0;
    }

    public void calculatePerformance() {
        double performanceScore = 0.0;
        int maxProjects = 5;
        int projectsCompleted = 0;
        int feedbackCount = 3;
        double[] feedbackScores = {8.0, 7.5, 9.0};
        double weightProjects = 0.4;

        int projectIndex = 0;
        while (projectIndex < maxProjects) {
            if (Math.random() > 0.3) {
                projectsCompleted++;
            }
            projectIndex++;
        }

        double totalFeedback = 0.0;
        for (int i = 0; i < feedbackCount; i++) {
            totalFeedback += feedbackScores[i];
        }
        double avgFeedback = totalFeedback / feedbackCount;
        if (performanceScore >= 85) {
            printPerformanceDetail("Excellent!");
        } else if (performanceScore >= 65) {
            printPerformanceDetail("Good!");
        } else if (performanceScore >= 45) {
            printPerformanceDetail("Average!");
        } else {
            printPerformanceDetail("Needs Improvement!");
        }
    }

    private void printPerformanceDetail(String message) {
        System.out.println(message);
    }
}

class Intern extends StaffMember {
    private int internshipDuration;

```

```

class StaffMember(Employee):
    def __init__(self, name, employee_id, base_salary):
        super().__init__(name, employee_id)
        self.base_salary = base_salary

    def calculatePayment(self, Experience):
        return self.base_salary + Experience * 100.0

    def calculatePerformance(self):
        performance_score = 0.0
        max_projects = 5
        projects_completed = 0
        feedback_count = 3
        feedback_scores = [8.0, 7.5, 9.0]
        weight_projects = 0.4

        project_index = 0
        while project_index < max_projects:
            if import_random.random() > 0.3:
                projects_completed += 1
            project_index += 1

        total_feedback = 0.0
        for i in range(0, feedback_count, 1):
            total_feedback += feedback_scores[i]

        avgFeedback = totalFeedback / feedbackCount
        if performance_score >= 85:
            self.print_performance_detail(f"Excellent!")
        elif performance_score >= 65:
            self.print_performance_detail(f"Good!")
        elif performance_score >= 45:
            self.print_performance_detail(f"Average!")
        else:
            self.print_performance_detail(f"Needs Improvement!")

    def print_performance_detail(self, message):
        print(message)

class Intern(StaffMember):
    def __init__(self, name, employee_id, base_salary, internship_duration):
        super().__init__(name, employee_id, base_salary)
        self.internship_duration = internship_duration

    def calculate_salary(self, stipend_adjustment):
        return super().calculate_salary() + stipend_adjustment

    def calculatePerformance(self):
        super().calculate_performance()

```

```

public Intern(String name, int employeeId, double
baseSalary, int internshipDuration) {
    super(name, employeeId, baseSalary);
    this.internshipDuration = internshipDuration;
}
public double calculateSalary(double stipendAdjustment) {
    return super.calculateSalary() + stipendAdjustment;
}
public void calculatePerformance() {
    super.calculatePerformance();
}
public String generateReport() {
    return "Name: " + name + ", Base Salary: " + baseSalary;
}
public void scheduleTraining(int numberOfSessions) {
    System.out.println("Session " + numberOfSessions);
}
public void submitProgressReport(String reportContent) {
    System.out.println(reportContent);
}
}

```

```

def generate_report(self):
    return f"Name: {self.name}, Base Salary: {self.base_salary}"

def schedule_training(self, number_of_sessions):
    print(f"Session {number_of_sessions}.")

def submit_progress_report(self, report_content):
    print(report_content)

```

Q3. [CO5] Identify the code smells from the given code below and refactor it correctly. [12]

[NB: You should mark the code segment and write the name of the identified code smell and the possible makeover of the smell in your question; you should write the refactored version of the given code.]

JAVA	PYTHON
<pre> import java.util.*; public class HumanSpaceshipControl { String user_eyecolor = "green"; public double [] boostFuelCells(double[] fuelCells, double minCharge, double boostFactor, double orbitOffset, double gravPull, double alienTechLevel, double wormholeBoost) { // These are info of ship boolean [] is_active = new boolean [fuelCells.length]; int activeCells = 0; // Activate only those fuel cells with charge above minimum for (int i = 0; i < fuelCells.length; i++) { if (fuelCells[i] > minCharge) { is_active[i] = true; activeCells++; } } } } </pre>	<pre> class HumanSpaceshipControl : def __init__(self): self.user_eyecolor = "green" def boostFuelCells(self, fuel_cells, min_charge, boost_factor, orbit_offset, grav_pull, alien_tech_level, wormhole_boost): # These are info of ship is_active = [False] * len(fuel_cells) active_cells = 0 # Activate only those fuel cells with charge above minimum for i in range(len(fuel_cells)): if fuel_cells[i] > min_charge: is_active[i] = True active_cells += 1 # Visiting fuel cells one by one and checking whether that fuel cell satisfies boost condition or not. If satisfies boost condition then we are boosting that cell total_boosted_power = 0 </pre> duplicate code Here we can add a checkFuel method. comment smell comment smell.

```

    }
    // Visiting fuel cells one by one and checking
    whether that fuel cell satisfies boost condition or not. If
    satisfies boost condition then we are boosting that cell
    double totalBoostedPower = 0;
    for (int i = 0; i < fuelCells.length; i++) {
        if (fuelCells[i] > boostFactor &&
is_active[i] == true && fuelCells[i] != wormholeBoost
&& user_eyecolor != "purple") {
            fuelCells[i] = fuelCells[i] + fuelCells[i] * boostFactor;
        }
    }
    System.out.println("Fuel cells energized for
hyperspace jump.");
    return fuelCells;
}

public boolean[] triggerFuelCells(double[]
fuelCells, double minCharge) {
    boolean[] isActive = new
boolean[fuelCells.length];
    int i = 0;
    while (i < fuelCells.length) {
        if (fuelCells[i] > minCharge) {
            isActive[i] = true;
        }
        i++;
    }
    return isActive;
}

public void launchSanda(double[] fuelCells, double
minCharge, double boostFactor, double orbitOffset,
double gravPull, double alienTechLevel, double
wormholeBoost) #Scanning fuel level
{
    boolean[] isActive = new
boolean[fuelCells.length];
    int index = 0;
    for (double charge : fuelCells) {
        if (charge > minCharge) {
            isActive[index] = true;
        }
        index++;
    }
    double totalEnergy = 0;
    double minChargeLevel = fuelCells[0];
    for (int i = 0; i < fuelCells.length; i++) {
        totalEnergy += fuelCells[i];
        if (fuelCells[i] < minChargeLevel &&
isActive[i]) minChargeLevel = fuelCells[i];
    }
    double avgEnergy = totalEnergy / fuelCells.length;
    System.out.println(avgEnergy, minChargeLevel);
}
}

```

```

for i in range(len(fuel_cells)):
    if (fuel_cells[i] > boost_factor and is_active[i] == True and
fuel_cells[i] != wormhole_boost and self.user_eyecolor != "purple"):

        fuel_cells[i] = fuel_cells[i] + fuel_cells[i] * boost_factor

    print("Fuel cells energized for hyperspace jump.")
    return fuel_cells

    wrong naming
def triggerFuelCells(self, fuel_cells, min_charge):
    is_active = [False] * len(fuel_cells)
    i = 0

    while i < len(fuel_cells):
        if fuel_cells[i] > min_charge:
            is_active[i] = True
        i += 1

    return is_active

    wrong naming of method
def launchSanda(self, fuel_cells, min_charge, boost_factor,
orbit_offset, grav_pull, alien_tech_level, wormhole_boost): #Scanning
fuel level

    is_active = [False] * len(fuel_cells)
    index = 0
    for charge in fuel_cells:
        if charge > min_charge:
            is_active[index] = True
            index += 1

    total_energy = 0
    min_charge_level = fuel_cells[0]

    for i in range(len(fuel_cells)):
        total_energy += fuel_cells[i]
        if fuel_cells[i] < min_charge_level and is_active[i]:
            min_charge_level = fuel_cells[i]
    avg_energy = total_energy / len(fuel_cells)
    print(avg_energy, min_charge_level)

    duplicate code

```